You said:

Complete array for AIML engineer basic to advanced concept explain
code frame like practical world for learning
make notes for revision and memorization

ChatGPT said:

Complete Python Arrays for an AIML Engineer — Basic to Advanced

A complete, practical study guide with simple explanations, Python code, dry runs, real-world AI/ML examples, interview questions, and revision notes.

For an AI/ML engineer, learning arrays means understanding **Python lists, the built-in `array` module, and NumPy arrays**. NumPy is especially important because it is used for numerical computation, data preprocessing, machine learning, deep learning, and scientific computing.

1. Your array learning roadmap

Level 1 — Fundamentals

What arrays are, indexing, slicing, traversal, updating, insertion, deletion, searching, and basic operations.

Level 2 — Python coding patterns

Reverse, sorting, frequency counts, duplicates, second-largest values, two pointers, prefix sums, and sliding windows.

Level 3 — NumPy arrays

ndarray, dimensions, shape, dtype, reshape, broadcasting, vectorization, Boolean masks, and mathematical operations.

Level 4 — AI/ML applications

Feature matrices, labels, normalization, image arrays, tensors, batches, and model input shapes.

Level 5 — Interview preparation

Coding patterns, dry runs, time complexity, common mistakes, interview questions, and quick revision.

2. What is an array?

An **array** is a data structure used to store multiple values in an organized way. You can access individual elements using their index.

For example, suppose you record the marks of five students.

```
marks = [78, 85, 92, 67, 88]

print(marks[0])  # 78
print(marks[2])  # 92
```

Indexing starts at `0` in Python sequences.

Visual: positive indexing

78	85	92	67	88
Index 0	Index 1	Index 2	Index 3	Index 4

Remember: first element = index 0; last element = index length − 1.

Array vs list vs NumPy array

Python beginners often use the word *array* to mean any collection of values. Technically, these structures are different.

Feature	Python list	`array.array`	NumPy `ndarray`
Built into Python	Yes	Yes, via module	No, install NumPy
Element types	Can be mixed	One declared type code	Usually one dtype per array
Multiple dimensions	Nested lists	Primarily one-dimensional	Yes
Numerical computation	Possible	Limited	Excellent
Importance for AI/ML	Useful	Occasional	Essential

Practical rule: Learn array algorithms using Python lists first, then learn NumPy for numerical and AI/ML work.

3. Python array fundamentals

3.1 Creating an array-like collection

```
# Integer values
numbers = [10, 20, 30, 40, 50]

# Strings
names = ["Aamir", "Rahul", "Priya"]

# Floating-point values
temperatures = [28.5, 30.2, 29.8]

# Mixed data types: valid in a Python list
data = [10, "Python", 3.14, True]

# Empty list
empty = []

print(numbers)
print(names)
print(len(numbers))  # 5
```

`len(numbers)` returns the number of elements.

3.2 Accessing elements using indexing

```
numbers = [10, 20, 30, 40, 50]

print(numbers[0])   # 10
print(numbers[1])   # 20
print(numbers[-1])  # 50
print(numbers[-2])  # 40
```

Memory trick

- Positive index: count from the left, starting at `0`.
- Negative index: count from the right, starting at `-1`.

- Invalid index: Python raises `IndexError`.

3.3 Slicing an array

Syntax:

```
array[start:stop:step]
```

The `start` index is included, but `stop` is excluded.

```
a = [10, 20, 30, 40, 50, 60]

print(a[1:4])   # [20, 30, 40]
print(a[:3])    # [10, 20, 30]
print(a[2:])    # [30, 40, 50, 60]
print(a[::2])   # [10, 30, 50]
print(a[::-1])  # [60, 50, 40, 30, 20, 10]
```

Dry run for `a[1:4]`:

Index	Value	Included?
0	10	No
1	20	Yes
2	30	Yes
3	40	Yes
4	50	No — stop index

Interview point: Slicing returns a new list. For a normal list, the slice is a shallow copy, not a deep copy.

3.4 Updating array elements

```
marks = [70, 80, 90]

marks[1] = 95

print(marks)
# [70, 95, 90]
```

Python lists are mutable, which means their elements can be changed after creation.

3.5 Adding and removing elements

```
numbers = [10, 20, 30]

numbers.append(40)        # Add one element at the end
numbers.insert(1, 15)     # Insert at index 1
numbers.extend([50, 60])  # Add multiple elements

print(numbers)
# [10, 15, 20, 30, 40, 50, 60]

numbers.remove(30)  # Remove first matching value
last = numbers.pop()  # Remove and return last element
del numbers[0]        # Delete element at index 0

print(numbers)
print(last)  # 60
```

Operation	Purpose	Typical time
`append(x)`	Add at end	$O(1)$ amortized
`insert(i, x)`	Insert at index	$O(n)$
`remove(x)`	Remove first matching value	$O(n)$
`pop()`	Remove last element	$O(1)$
`pop(i)`	Remove at index	$O(n)$
`extend(iterable)`	Add several values	$O(k)$ amortized for k added elements

Remember that `remove()` accepts a value, while `pop()` accepts an index and returns the removed element.

4. Traversing and searching arrays

Traversal means visiting the elements one by one.

Method 1: Direct traversal

```
numbers = [10, 20, 30, 40]

for value in numbers:
    print(value)
```

Use this when you need only the values.

Method 2: Index-based traversal

```
numbers = [10, 20, 30, 40]

for i in range(len(numbers)):
    print(i, numbers[i])
```

Use this when you need both indexes and values.

Method 3: `enumerate()`

```
numbers = [10, 20, 30, 40]

for index, value in enumerate(numbers):
    print(index, value)
```

`enumerate()` is usually the clearest choice when you need both.

Linear search — find a value

Practical example: find a particular student's mark.

```
marks = [65, 82, 91, 74, 88]
target = 91

found_index = -1

for i, mark in enumerate(marks):
    if mark == target:
        found_index = i
        break
```

```
print(found_index)  # 2
```

Dry run:

Step	Current value	Comparison	Result
1	65	65 == 91?	Continue
2	82	82 == 91?	Continue
3	91	91 == 91?	Found; stop

Time complexity: $O(n)$ in the worst case.

5. Essential built-in operations

```
a = [12, 5, 30, 8, 15]

print(len(a))       # 5
print(min(a))       # 5
print(max(a))       # 30
print(sum(a))       # 70
print(sorted(a))    # [5, 8, 12, 15, 30]
print(a.count(5))   # 1
print(a.index(30))  # 2
print(8 in a)       # True
```

Important distinction:

```
a = [3, 1, 2]

result = sorted(a)
print(result)  # [1, 2, 3]
print(a)       # [3, 1, 2]

a.sort()
print(a)       # [1, 2, 3]
```

- `sorted(a)` returns a new sorted list.
- `a.sort()` sorts the original list in place and returns `None`.

---

6. Core array coding problems — interview practice

These patterns are useful for coding interviews and help build problem-solving skills for data science.

Problem 1: Find the sum and average

```
marks = [80, 90, 70, 100, 60]

total = sum(marks)
average = total / len(marks)

print("Total:", total)      # 400
print("Average:", average)  # 80.0
```

Formula:

$$\text{Average} = \frac{\text{Sum of elements}}{\text{Number of elements}}$$

Always handle an empty list before dividing by its length.

Problem 2: Find the largest element without `max()`

```
a = [12, 45, 7, 89, 23]

largest = a[0]

for value in a[1:]:
    if value > largest:
        largest = value

print(largest)  # 89
```

Dry run:

Current value	Largest before	Largest after
45	12	45
7	45	45

89	45	89
23	89	89

Time complexity: $O(n)$.

Problem 3: Find the second-largest distinct value

```
a = [10, 20, 5, 20, 15]

unique_values = set(a)
ordered = sorted(unique_values)

if len(ordered) >= 2:
    print(ordered[-2])  # 15
else:
    print("No second-largest distinct value")
```

This is simple and readable. Its time complexity is $O(n \log n)$.

For a single-pass solution:

```
a = [10, 20, 5, 20, 15]

largest = None
second = None

for value in a:
    if largest is None or value > largest:
        second = largest
        largest = value
    elif value != largest and (
        second is None or value > second
    ):
        second = value

print(second)  # 15
```

This version takes $O(n)$ time and $O(1)$ auxiliary space.

Problem 4: Reverse an array

```
a = [1, 2, 3, 4, 5]

reversed_a = a[::-1]

print(reversed_a)  # [5, 4, 3, 2, 1]
```

For an in-place reversal:

```
a.reverse()
print(a)  # [5, 4, 3, 2, 1]
```

Problem 5: Count even and odd numbers

```
a = [10, 15, 22, 33, 40, 51]

even_count = 0
odd_count = 0

for value in a:
    if value % 2 == 0:
        even_count += 1
    else:
        odd_count += 1

print("Even:", even_count)  # 3
print("Odd:", odd_count)    # 3
```

Memory trick: `% 2 == 0` means even; `% 2 != 0` means odd.

Problem 6: Remove duplicates while preserving order

```
a = [1, 2, 2, 3, 1, 4, 3]

result = []

for value in a:
    if value not in result:
        result.append(value)
```

```
print(result)  # [1, 2, 3, 4]
```

This beginner-friendly version can take $O(n^2)$ time because membership testing in a list is linear.

A faster average-case approach:

```
a = [1, 2, 2, 3, 1, 4, 3]

result = list(dict.fromkeys(a))

print(result)  # [1, 2, 3, 4]
```

Problem 7: Frequency of each element

Practical example: count how often each predicted class appears in a dataset.

```
labels = ["cat", "dog", "cat", "bird", "dog", "cat"]

frequency = {}

for label in labels:
    frequency[label] = frequency.get(label, 0) + 1

print(frequency)
# {'cat': 3, 'dog': 2, 'bird': 1}
```

`dict.get(key, default)` returns the existing value or the default if the key is missing.

Problem 8: Find common elements between two arrays

```
a = [1, 2, 3, 4, 5]
b = [3, 4, 5, 6, 7]

common = list(set(a) & set(b))

print(sorted(common))  # [3, 4, 5]
```

Sets remove duplicates and provide efficient average-case membership operations. If order or repeated occurrences matter, use a different approach.

7. Advanced array problem-solving patterns

Once you understand basic traversal and searching, learn these patterns. They are useful for coding interviews and many data-processing tasks.

1. Two pointers

Use two indexes that move through the array, often from opposite ends. Useful for reversing, sorted pair-sum problems, and palindrome checks.

2. Prefix sums

Store cumulative totals so that range sums can be calculated quickly after preprocessing.

3. Sliding window

Move a window over consecutive elements. Useful for maximum sums of fixed-size subarrays and rolling statistics.

4. Binary search

Repeatedly halve a sorted search range. Time complexity is $O(\log n)$.

7.1 Two pointers: pair sum in a sorted array

Find two values that add up to a target.

```
a = [1, 2, 4, 6, 8, 9]
target = 10

left = 0
right = len(a) - 1

while left < right:
    current_sum = a[left] + a[right]

    if current_sum == target:
        print(a[left], a[right])
        break
    elif current_sum < target:
        left += 1
    else:
        right -= 1
```

Output:

```
2 8
```

Why it works: the array is sorted. If the sum is too small, increase the left value; if it is too large, decrease the right value.

Complexity: $O(n)$ time and $O(1)$ auxiliary space.

7.2 Prefix sum: fast range calculations

```
a = [2, 4, 6, 8, 10]

prefix = [0]

for value in a:
    prefix.append(prefix[-1] + value)

# Sum from index 1 through index 3, inclusive
left = 1
right = 3

range_sum = prefix[right + 1] - prefix[left]

print(prefix)     # [0, 2, 6, 12, 20, 30]
print(range_sum)  # 18
```

Formula:

$$\text{sum}(l, r) = P[r + 1] - P[l]$$

Here, $P[k]$ represents the sum of the first k elements. Preprocessing takes $O(n)$; each range-sum query then takes $O(1)$.

7.3 Sliding window: maximum sum of three consecutive elements

```
a = [2, 1, 5, 1, 3, 2]
k = 3

window_sum = sum(a[:k])
```

```
max_sum = window_sum

for i in range(k, len(a)):
    window_sum += a[i] - a[i - k]
    max_sum = max(max_sum, window_sum)

print(max_sum)  # 9
```

Dry run:

Window	Values	Sum
1	2, 1, 5	8
2	1, 5, 1	7
3	5, 1, 3	9
4	1, 3, 2	6

The maximum is `9`. After initialization, each window is updated in constant time.

7.4 Binary search in a sorted array

```
a = [10, 20, 30, 40, 50, 60]
target = 50

left = 0
right = len(a) - 1
answer = -1

while left <= right:
    mid = (left + right) // 2

    if a[mid] == target:
        answer = mid
        break
    elif a[mid] < target:
        left = mid + 1
    else:
        right = mid - 1

print(answer)  # 4
```

Important: Binary search requires sorted data (or another structure with the necessary ordering property). It takes $O(\log n)$ time.

---

8. NumPy arrays — the most important section for AIML

NumPy is a Python library for numerical computing. Its central structure is the `ndarray`, which supports efficient operations on one-dimensional and multidimensional arrays.

Install it once in your Python environment:

```
python -m pip install numpy
```

Import it:

```
import numpy as np
```

8.1 Creating NumPy arrays

```
import numpy as np

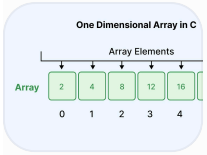
a = np.array([10, 20, 30, 40])
b = np.zeros(4)
c = np.ones(4)
d = np.arange(0, 10, 2)
e = np.linspace(0, 1, 5)

print(a)  # [10 20 30 40]
print(b)  # [0. 0. 0. 0.]
print(c)  # [1. 1. 1. 1.]
print(d)  # [0 2 4 6 8]
print(e)  # [0.   0.25 0.5  0.75 1.  ]
```

Remember the difference:

- `np.arange(start, stop, step)` uses a step size; the stop value is excluded.
- `np.linspace(start, stop, num)` generates a specified number of evenly spaced values; the stop value is included by default.

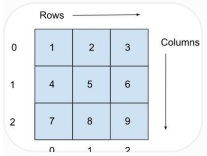
8.2 Dimensions: 1D, 2D, and 3D



1D array — vector

```
a = np.array([10, 20, 30])
print(a.shape)  # (3,)
print(a.ndim)   # 1
```

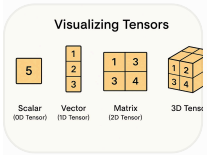
Three elements in a single dimension.



2D array — matrix

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])
print(a.shape)  # (2, 3)
print(a.ndim)   # 2
```

Two rows and three columns.



3D array — a common tensor shape

```
a = np.zeros((2, 3, 4))
print(a.shape)  # (2, 3, 4)
print(a.ndim)   # 3
```

Two groups, each with three rows and four columns.

In machine learning, a 2D array often represents rows of samples and columns of features. In deep learning, 3D and 4D arrays frequently represent sequences, batches, or images.

8.3 Shape, size, and data type

```
import numpy as np

X = np.array([[1, 2, 3],
              [4, 5, 6]])

print(X.shape)  # (2, 3)
print(X.ndim)   # 2
print(X.size)   # 6
print(X.dtype)  # commonly int64 or int32
```

Property	Meaning
`shape`	Number of elements along each dimension
`ndim`	Number of dimensions
`size`	Total number of elements
`dtype`	Data type of array elements
`itemsize`	Bytes occupied by one element

The exact integer dtype depends on how the array was created and the platform.

8.4 Reshaping arrays

```
a = np.arange(1, 7)

matrix = a.reshape(2, 3)

print(matrix)
# [[1 2 3]
#  [4 5 6]]
```

You can also use `-1` to let NumPy infer one dimension:

```
a = np.arange(12)

print(a.reshape(3, -1).shape)  # (3, 4)
print(a.reshape(-1, 2).shape)  # (6, 2)
```

Rule to memorize: Reshaping cannot change the total number of elements.

$$\text{Total elements} = \prod_{i} \text{shape}_i$$

For example, 12 elements can be reshaped into `(3, 4)`, `(2, 6)`, or `(12,)`, but not `(5, 3)`.

8.5 Indexing and slicing in NumPy

```
import numpy as np

X = np.array([[10, 20, 30],
              [40, 50, 60],
              [70, 80, 90]])

print(X[0, 1])   # 20
print(X[1, 2])   # 60
print(X[0, :])   # [10 20 30]
print(X[:, 1])   # [20 50 80]
print(X[:2, :2])
# [[10 20]
#  [40 50]]
```

Remember the pattern `X[row, column]`. A colon selects a range along that axis.

8.6 Vectorization: why NumPy is useful

Vectorization means applying an operation to an entire array without writing an explicit Python loop for every element.

```
import numpy as np

marks = np.array([60, 70, 80, 90])

adjusted_marks = marks + 5
percentages = marks / 100

print(adjusted_marks)  # [65 75 85 95]
print(percentages)    # [0.6 0.7 0.8 0.9]
```

With a Python list, `marks + 5` raises `TypeError`. With a NumPy array, it adds five to every element.

NumPy operations often run much faster for large numerical datasets because their implementation uses optimized compiled code. Actual performance depends on the operation and array size.

8.7 Broadcasting

Broadcasting allows NumPy to perform operations on compatible shapes without manually repeating the smaller array.

```
import numpy as np

X = np.array([[10, 20, 30],
              [40, 50, 60]])

bias = np.array([1, 2, 3])

print(X + bias)
# [[11 22 33]
#  [41 52 63]]
```

The shape of `X` is `(2, 3)` and the shape of `bias` is `(3,)`. NumPy applies the bias to each row.

Memory rule: compare shapes from right to left. Dimensions are compatible when they are equal or one of them is `1` (missing leading dimensions are treated as `1`).

8.8 Boolean masking and filtering

This is useful for selecting data points that meet a condition.

```
import numpy as np

scores = np.array([45, 70, 85, 32, 90])

mask = scores >= 60

print(mask)
# [False  True  True False  True]

print(scores[mask])
# [70 85 90]
```

A shorter form:

```
passed = scores[scores >= 60]
print(passed)  # [70 85 90]
```

To combine conditions, use `&` for AND and `|` for OR, with parentheses around each comparison:

```
scores = np.array([45, 70, 85, 32, 90])

selected = scores[(scores >= 60) & (scores < 90)]

print(selected)  # [70 85]
```

8.9 Aggregations and axis

```
import numpy as np

X = np.array([[10, 20, 30],
              [40, 50, 60]])

print(np.sum(X))         # 210
print(np.mean(X))        # 35.0
print(np.sum(X, axis=0)) # [50 70 90]
print(np.sum(X, axis=1)) # [ 60 150]
```

For a 2D array:

- `axis=0` reduces the rows, giving one result per column.
- `axis=1` reduces the columns, giving one result per row.

This is an important interview topic and a common source of confusion.

9. Real-world AIML practical: student performance prediction

Let's create a small dataset where each row represents a student and each column represents a feature.

Dataset visualization

Rows = students; columns = study hours, attendance, and assignments completed.

Study hours	Attendance %	Assignments
2	60	3
5	80	7

8	95	10

Step 1: Represent features and labels

```
import numpy as np

# Feature matrix: 3 students, 3 features
X = np.array([
    [2, 60, 3],
    [5, 80, 7],
    [8, 95, 10]
], dtype=float)

# Target: actual exam marks
y = np.array([50, 75, 92])

print("Features shape:", X.shape)  # (3, 3)
print("Labels shape:", y.shape)    # (3,)
```

In supervised machine learning:

- `X` represents input features.
- `y` represents target values.
- Each row of `X` corresponds to one observation.
- Each value of `y` corresponds to the matching row in `X`.

Step 2: Normalize the features

Features often have different scales. Study hours range from 2 to 8, while attendance ranges from 60 to 95.

Min-max scaling maps values to the interval 0 to 1.

$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

```
minimum = X.min(axis=0)
maximum = X.max(axis=0)

X_scaled = (X - minimum) / (maximum - minimum)
```

```
print(np.round(X_scaled, 2))
# [[0.   0.   0.  ]
#  [0.5  0.57 0.57]
#  [1.   1.   1.  ]]
```

For real ML training, calculate the minimum and maximum from the **training set only**, then reuse those same values to transform validation, test, and future data. This avoids data leakage.

Step 3: Calculate statistics

```
print("Average study hours:", np.mean(X[:, 0]))
print("Average attendance:", np.mean(X[:, 1]))
print("Average marks:", np.mean(y))
```

This small example teaches array handling and preprocessing. It is not yet a trained prediction model; that requires splitting the data and fitting an appropriate estimator.

10. Images, tensors, and deep learning

Arrays are also used to represent images and batches of training data.

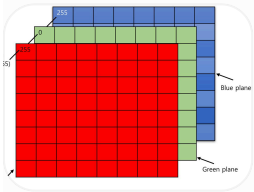


Image array

A color image is commonly represented as `(height, width, 3)`, where the last dimension contains red, green, and blue channel values.

```
import numpy as np

# Simulated 64 × 64 RGB image
image = np.zeros((64, 64, 3), dtype=np.uint8)

# Make one pixel red
image[10, 20] = [255, 0, 0]

print(image.shape)  # (64, 64, 3)
print(image[10, 20])  # [255   0   0]
```

A tensor is a general multidimensional numerical structure. NumPy arrays are commonly used to prepare numerical data; deep-learning frameworks such as PyTorch and TensorFlow provide their own tensor types.

For a batch of 32 RGB images, each of size 64 × 64, the batch might have shape:

- `(32, 64, 64, 3)` for channels-last data.
- `(32, 3, 64, 64)` for channels-first data.

The required format depends on the model and framework.

11. Important NumPy concepts to memorize

Concept	Meaning	Example
`ndarray`	Multidimensional array	`np.array([1, 2])`
`shape`	Dimensions	`(3, 4)`
`ndim`	Number of dimensions	`X.ndim`
`dtype`	Element type	`X.dtype`
`reshape()`	Change shape without changing element count	`X.reshape(2, 3)`
`axis=0`	Reduce rows; operate per column	`np.sum(X, axis=0)`
`axis=1`	Reduce columns; operate per row	`np.sum(X, axis=1)`
Broadcasting	Operate on compatible shapes	`X + bias`
Vectorization	Array-wide numerical operations	`X * 2`
Boolean mask	Filter using conditions	`X[X > 5]`
`np.concatenate()`	Join arrays along an existing axis	`np.concatenate((a, b))`
`np.stack()`	Combine arrays along a new axis	`np.stack((a, b))`
`np.dot()` / `@`	Dot product or matrix multiplication, depending on dimensions	`A @ B`

Matrix multiplication example

```
import numpy as np

A = np.array([[1, 2],
              [3, 4]])

B = np.array([[5, 6],
```

```
              [7, 8]])

print(A @ B)
# [[19 22]
#  [43 50]]
```

Matrix multiplication requires the number of columns in `A` to equal the number of rows in `B`. For example, `(2, 3) @ (3, 4)` produces a `(2, 4)` result.

View vs copy — an advanced interview topic

```
import numpy as np

a = np.array([10, 20, 30, 40])

view = a[1:3]
view[0] = 99

print(a)  # [10 99 30 40]
```

Basic NumPy slicing generally returns a **view** that shares the original array's data. Changing the view can therefore change the original.

To make an independent copy:

```
a = np.array([10, 20, 30, 40])

copied = a[1:3].copy()
copied[0] = 99

print(a)       # [10 20 30 40]
print(copied)  # [99 30]
```

Do not assume every NumPy operation returns a view or every operation returns a copy; the behavior depends on the operation.

12. Common mistakes and how to avoid them

⚠ **Mistake 1: Confusing indexes and values**

`a[2]` means the element at index 2, not the second element. The second element is `a[1]`.

Mistake 2: Expecting list arithmetic to work like NumPy

`[1, 2] * 2` produces `[1, 2, 1, 2]`, while `np.array([1, 2]) * 2` produces `[2, 4]`.

Mistake 3: Using `axis` incorrectly

For a 2D array, `axis=0` gives column results and `axis=1` gives row results.

Mistake 4: Reshaping incompatible sizes

Six elements cannot be reshaped into `(2, 4)` because that shape requires eight elements.

Mistake 5: Data leakage in ML preprocessing

Never calculate scaling statistics from the entire dataset before splitting. Fit preprocessing on training data only.

13. Array interview questions with answers

1. What is an array?
2. What is the difference between a list and a NumPy array?
3. What is time complexity?
4. What is slicing?
5. What is vectorization?
6. What is broadcasting?

7. What does shape (100, 5) mean in ML?

8. What is the difference between reshape and resize?

9. What is the difference between a view and a copy?

10. Why are arrays important in deep learning?

14. Practice exercises — test your knowledge

Try these without looking at the answers first.

Your practice checklist 0/6

- [ ] Find the smallest value without using min().

Start with the first element and compare every remaining value.

```
a = [8, 3, 12, 1, 7]
# Expected: 1
```

- [ ] Move all zeros to the end while preserving other values.

Collect nonzero values, then append the required zeros.

```
a = [0, 1, 0, 3, 12]
# Expected: [1, 3, 12, 0, 0]
```

- [ ] Find all values greater than the average.

Calculate the mean, then filter the array.

```
a = [10, 20, 30, 40]
# Expected: [30, 40]
```

- [ ] Create a 3 × 3 NumPy identity matrix.

Use np.eye().

```
# Expected: diagonal values are 1; other values are 0.
```

- [ ] Standardize a NumPy array using mean and standard deviation.

Use (X - X.mean()) / X.std() for a nonconstant array.

X = np.array([1., 2., 3.])
Expected: [-1.2247, 0, 1.2247] approximately

Select the second column of a 2D array.

Use a colon for all rows and index 1 for the second column.

X[:, 1]

Show complete solutions with dry runs

15. Final revision notes — memorize these

Python array cheat sheet

Review this before interviews or practical coding.

Python list

```
a = [10, 20, 30, 40]
a[0]          # Access
a[-1]         # Last element
a[1:3]        # Slice
a.append(50)  # Add
a.pop()       # Remove last
len(a)        # Length
sorted(a)     # New sorted list
a[::-1]       # Reversed copy
```

NumPy

```
import numpy as np

X = np.array([[1, 2], [3, 4]])
X.shape        # (2, 2)
X.ndim         # 2
X.size         # 4
X.reshape(4)   # One-dimensional view/array
X.mean()       # Mean
X.sum(axis=0)  # Column sums
```

```
X.sum(axis=1)  # Row sums
X[X > 2]       # Boolean filtering
```

Complexity to remember

Operation	Typical complexity
Index access in a list	$O(1)$
Linear search	$O(n)$
Binary search	$O(\log n)$
Sorting	$O(n \log n)$
Full traversal	$O(n)$
List insertion at the beginning	$O(n)$

Five memory rules

1. Indexing starts at zero.
2. Slice stop is excluded.
3. `append()` adds one element; `extend()` adds elements from an iterable.
4. NumPy `shape` tells you the dimensions; `size` tells you the total element count.
5. In ML, rows commonly represent samples and columns represent features.

Recommended learning order: Python list fundamentals → array coding problems → time complexity → NumPy fundamentals → broadcasting and vectorization → ML preprocessing → tensors and deep learning.

Do not just memorize code. For every problem, practice explaining the input, output, algorithm, dry run, edge cases, and time complexity. That is what turns array knowledge into practical AIML engineering skill.

You said:

give me problem for aiml engineer and solution also

ChatGPT said:

Python Array Problems for AIML Engineers — With Solutions

Here is a practical problem-solving workbook from beginner to advanced level. These problems will help you prepare for **AI/ML interviews, Python coding rounds, data preprocessing, NumPy, and real-world machine learning tasks**.

For each problem, focus on four things: understand the problem, write the code, dry-run it, and explain its time complexity.

Level 1 — Python array fundamentals

Problem 1. Calculate the mean of student marks

Real-world use: Calculate the average score in a dataset.

Input: `[70, 80, 90, 60, 100]`

Expected output: `80.0`

Solution

```
marks = [70, 80, 90, 60, 100]

total = 0

for mark in marks:
    total += mark

average = total / len(marks) if marks else 0

print("Total:", total)
print("Average:", average)
```

Output:

```
Total: 400
Average: 80.0
```

Dry run:

Mark	Total after addition

70	70
80	150
90	240
60	300
100	400

- Time complexity: $O(n)$
- Space complexity: $O(1)$

Problem 2. Find the minimum and maximum values

Real-world use: Find the lowest and highest recorded temperature in a dataset.

```
temperatures = [32, 28, 35, 24, 40, 30]

smallest = temperatures[0]
largest = temperatures[0]

for value in temperatures[1:]:
    if value < smallest:
        smallest = value

    if value > largest:
        largest = value

print("Minimum:", smallest)
print("Maximum:", largest)
```

Output:

```
Minimum: 24
Maximum: 40
```

Why it works: keep track of the smallest and largest values seen so far as you traverse the array.

- Time: $O(n)$
- Extra space: $O(1)$

Problem 3. Count missing or invalid values

Real-world use: Data cleaning before training an ML model.

```
data = [10, None, 25, None, 40, 50]

missing_count = 0

for value in data:
    if value is None:
        missing_count += 1

cleaned_data = [value for value in data if value is not None]

print("Missing values:", missing_count)
print("Cleaned data:", cleaned_data)
```

Output:

```
Missing values: 2
Cleaned data: [10, 25, 40, 50]
```

Important: real datasets can also contain `NaN`, empty strings, or special missing-value markers. `None` checks alone do not handle all of them.

Problem 4. Normalize data using min-max scaling

Real-world use: Scale numerical features to a common range.

Formula:

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

```
data = [10, 20, 30, 40, 50]

minimum = min(data)
maximum = max(data)

if maximum == minimum:
    normalized = [0.0] * len(data)
else:
```

```
    normalized = [
        (x - minimum) / (maximum - minimum)
        for x in data
    ]

print(normalized)
```

Output:

```
[0.0, 0.25, 0.5, 0.75, 1.0]
```

Dry run:

Original	Calculation	Scaled
10	(10 − 10) / 40	0.00
20	(20 − 10) / 40	0.25
30	(30 − 10) / 40	0.50
40	(40 − 10) / 40	0.75
50	(50 − 10) / 40	1.00

For a real ML project, learn scaling parameters from the training set only and reuse them on validation and test data.

Level 2 — Array coding interview problems

Problem 5. Find the second-largest distinct element

Input: `[10, 20, 5, 20, 15]`

Output: `15`

```
a = [10, 20, 5, 20, 15]

largest = None
second_largest = None

for x in a:
    if largest is None or x > largest:
```

```
        second_largest = largest
        largest = x
    elif x != largest and (
        second_largest is None or x > second_largest
    ):
        second_largest = x

print(second_largest)
```

Dry run:

Value	Largest	Second largest
10	10	None
20	20	10
5	20	10
20	20	10
15	20	15

Time: $O(n)$. Extra space: $O(1)$.

If the array has fewer than two distinct values, the result is `None`.

Problem 6. Move all zeros to the end

Real-world use: Rearrange sparse or padded numerical data while preserving nonzero values.

Input: `[0, 1, 0, 3, 12]`

Output: `[1, 3, 12, 0, 0]`

```
a = [0, 1, 0, 3, 12]

result = []

for x in a:
    if x != 0:
        result.append(x)

zero_count = len(a) - len(result)
result.extend([0] * zero_count)
```

```
print(result)
```

Dry run:

1. Collect nonzero values: `[1, 3, 12]`
2. Count zeros: `5 - 3 = 2`
3. Append two zeros.
4. Final result: `[1, 3, 12, 0, 0]`

Time: $O(n)$. Extra space: $O(n)$.

Problem 7. Find duplicate values

Real-world use: Detect duplicate IDs or repeated records in a dataset.

```
ids = [101, 102, 103, 102, 104, 101, 105]

seen = set()
duplicates = set()

for value in ids:
    if value in seen:
        duplicates.add(value)
    else:
        seen.add(value)

print(sorted(duplicates))
```

Output:

```
[101, 102]
```

Time: $O(n)$ average. Extra space: $O(n)$.

Problem 8. Calculate the frequency of each value

Real-world use: Count class labels in a classification dataset.

```
labels = [1, 0, 1, 2, 0, 1, 2, 2, 0]

frequency = {}
```

```
for label in labels:
    frequency[label] = frequency.get(label, 0) + 1

print(frequency)
```

Output:

```
{1: 3, 0: 3, 2: 3}
```

This helps identify class distribution. In a real classification dataset, unequal label frequencies may indicate class imbalance, which should be investigated before choosing evaluation metrics or resampling strategies.

Problem 9. Find two numbers that add up to a target

Real-world use: Pair matching, numerical analysis, and interview problem-solving.

Input: `[2, 7, 11, 15]`, target = `9`

Output: indexes `(0, 1)`

```
a = [2, 7, 11, 15]
target = 9

seen = {}

for i, x in enumerate(a):
    complement = target - x

    if complement in seen:
        print("Indexes:", seen[complement], i)
        print("Values:", complement, x)
        break

    seen[x] = i
```

Output:

```
Indexes: 0 1
Values: 2 7
```

The dictionary stores previously encountered values and their indexes.

- Time: $O(n)$ average.
- Extra space: $O(n)$.

Problem 10. Calculate a moving average

Real-world use: Smooth noisy sensor readings or monitor rolling sales.

Input: `[10, 20, 30, 40, 50]`, window size = `3`

Output: `[20.0, 30.0, 40.0]`

```
data = [10, 20, 30, 40, 50]
k = 3

window_sum = sum(data[:k])
averages = [window_sum / k]

for i in range(k, len(data)):
    window_sum += data[i] - data[i - k]
    averages.append(window_sum / k)

print(averages)
```

Dry run:

Window	Sum	Average
10, 20, 30	60	20
20, 30, 40	90	30
30, 40, 50	120	40

Time: $O(n)$. Extra space for output: $O(n)$.

Level 3 — NumPy problems for AI/ML engineers

Install NumPy with `python -m pip install numpy` if needed.

Problem 11. Calculate statistics for a dataset

Real-world use: Summarize model features before training.

```
import numpy as np

ages = np.array([20, 25, 30, 35, 40])

print("Mean:", np.mean(ages))
print("Median:", np.median(ages))
print("Standard deviation:", np.std(ages))
print("Minimum:", np.min(ages))
print("Maximum:", np.max(ages))
```

Expected output:

```
Mean: 30.0
Median: 30.0
Standard deviation: 7.0710678118654755
Minimum: 20
Maximum: 40
```

By default, `np.std()` uses population standard deviation (`ddof=0`). For sample standard deviation, use `np.std(ages, ddof=1)`.

Problem 12. Select features using Boolean masking

Real-world use: Filter customers, sensor readings, or students according to a condition.

```
import numpy as np

scores = np.array([45, 78, 92, 55, 88, 35])

passed = scores[scores >= 60]

print(passed)
# [78 92 88]
```

Filter scores between 60 and 90, inclusive:

```
selected = scores[(scores >= 60) & (scores <= 90)]
```

```
print(selected)
# [78 88]
```

Memory trick: use `&` and `|` for elementwise conditions on NumPy arrays, and put parentheses around each comparison.

Problem 13. Standardize a feature

Real-world use: Prepare features for models that benefit from standardized inputs.

Formula:

$$z = \frac{x - \mu}{\sigma}$$

```
import numpy as np

X = np.array([10.0, 20.0, 30.0, 40.0, 50.0])

mean = X.mean()
std = X.std()

Z = (X - mean) / std

print(np.round(Z, 2))
```

Output:

```
[-1.41 -0.71  0.    0.71  1.41]
```

After standardization, the values have mean approximately zero and population standard deviation approximately one, provided the feature has nonzero variance.

For production ML, use a fitted preprocessing tool such as `sklearn.preprocessing.StandardScaler` and fit it only on the training data.

Problem 14. Create a feature matrix and extract columns

Real-world use: Prepare a tabular dataset for a model.

```
import numpy as np

# Columns: hours studied, attendance, assignments completed
X = np.array([
    [2, 60, 3],
    [5, 80, 7],
    [8, 95, 10]
])

study_hours = X[:, 0]
attendance = X[:, 1]
assignments = X[:, 2]

print("Shape:", X.shape)
print("Study hours:", study_hours)
print("Attendance:", attendance)
```

Output:

```
Shape: (3, 3)
Study hours: [2 5 8]
Attendance: [60 80 95]
```

Dry run:

- `X.shape` is `(3, 3)`: three samples, three features.
- `X[:, 0]` selects every row in the first column.
- `X[1, :]` selects all features for the second student.

Problem 15. Calculate the dot product

Real-world use: A simple model's weighted sum of input features.

```
import numpy as np

features = np.array([5.0, 80.0, 7.0])
weights = np.array([2.0, 0.1, 3.0])

prediction_score = np.dot(features, weights)

print(prediction_score)
```

Calculation:

$$(5 \times 2) + (80 \times 0.1) + (7 \times 3) = 39$$

Output:

```
39.0
```

This demonstrates a weighted sum, not a trained prediction model. Real models learn their weights from data and may also include a bias term.

Level 4 — Advanced AIML practical problems

Problem 16. Handle missing numerical values

Real-world use: Clean sensor readings before model training.

Suppose missing readings are represented by `np.nan`. Replace them with the mean of the available values.

```
import numpy as np

data = np.array([10.0, 20.0, np.nan, 40.0, np.nan])

valid_mean = np.nanmean(data)

cleaned = np.where(np.isnan(data), valid_mean, data)

print("Mean:", valid_mean)
print("Cleaned:", cleaned)
```

Output:

```
Mean: 23.333333333333332
Cleaned: [10.         20.         23.33333333 40.         23.33333333]
```

`np.nanmean()` ignores `NaN` values when calculating the mean. In real ML pipelines, fit imputation values on the training data only.

Problem 17. Calculate row-wise and column-wise statistics

Real-world use: Analyze multiple samples with multiple features.

```
import numpy as np

X = np.array([
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90]
])

print("Row means:", X.mean(axis=1))
print("Column means:", X.mean(axis=0))
```

Output:

```
Row means: [20. 50. 80.]
Column means: [40. 50. 60.]
```

Remember:

- `axis=0`: one result per column.
- `axis=1`: one result per row.

Problem 18. Compute Euclidean distance

Real-world use: Compare numerical feature vectors in clustering and nearest-neighbor algorithms.

Given two points, $A = (1, 2)$ and $B = (4, 6)$, calculate their distance.

```
import numpy as np

A = np.array([1, 2])
B = np.array([4, 6])

distance = np.sqrt(np.sum((A - B) ** 2))

print(distance)  # 5.0
```

Formula:

$$d(A, B) = \sqrt{\sum_i (A_i - B_i)^2}$$

The squared differences are 9 and 16. Their sum is 25, and the square root is 5.

Problem 19. Normalize each row of a matrix

Real-world use: Normalize feature vectors in some similarity-search and machine-learning applications.

```
import numpy as np

X = np.array([
    [3.0, 4.0],
    [5.0, 12.0]
])

norms = np.linalg.norm(X, axis=1, keepdims=True)

X_normalized = np.divide(
    X,
    norms,
    out=np.zeros_like(X),
    where=norms != 0
)

print(X_normalized)
```

Output:

```
[[0.6 0.8]
 [0.38461538 0.92307692]]
```

Each nonzero row is divided by its Euclidean norm. `keepdims=True` preserves a column shape so broadcasting works correctly. The code also handles zero rows without division by zero.

Problem 20. Calculate predictions for a batch of samples

Real-world use: Perform the forward calculation for a simple linear model.

```
import numpy as np

# Three samples, two features
X = np.array([
    [1.0, 2.0],
    [3.0, 4.0],
    [5.0, 6.0]
])

weights = np.array([0.5, 2.0])
bias = 1.0

predictions = X @ weights + bias

print(predictions)
```

Output:

```
[ 5.5 10.5 15.5]
```

The model computes:

$$\hat{y} = Xw + b$$

For the first sample, $(1 \times 0.5) + (2 \times 2) + 1 = 5.5$. NumPy calculates the same operation for every row at once.

21. Your AIML array interview checklist

Revision tracker

0/24 completed

1. Python fundamentals

- [ ] Indexing and slicing
- [ ] Append vs extend vs insert
- [ ] Remove vs pop vs del
- [ ] Reverse and sort arrays
- [ ] Find second-largest distinct element

- Frequency counts and duplicates

2. Coding patterns

- Linear search and binary search
- Two pointers
- Prefix sums
- Sliding window
- Pair sum with a dictionary
- Time and space complexity

3. NumPy essentials

- ndarray, shape, ndim, size, dtype
- Reshape and transpose
- Indexing and Boolean masks
- Vectorization and broadcasting
- Axis 0 vs axis 1
- Views vs copies and matrix multiplication

4. AIML applications

- Missing-value imputation
- Min-max scaling and standardization
- Feature matrices and labels
- Mean, standard deviation and distance
- Image arrays and tensor shapes
- Train-only preprocessing to prevent data leakage

Start a mock interview

My recommendation: Practice Problems 1–10 until you can code them without notes. Then master Problems 11–20 using NumPy. For every solution, explain why the algorithm works, what happens with empty input or duplicates, and how its time complexity changes with

dataset size. This is a strong foundation for Python and AIML engineering interviews.